DISEÑO DE SISTEMAS CON IA - UNIDAD 5 - TEORÍA
Agentes Inteligentes y Orquestación

Gracias al asistente implementado por Ortelana Textil, distribuidores y empleados comenzaron a resolver rápidamente consultas sobre productos, disponibilidad, políticas comerciales y reglamentos internos utilizando lenguaje natural. La reducción en los tiempos de respuesta, la disminución de errores humanos y la mejora en la satisfacción de los clientes impulsaron a la empresa a incorporar esta solución en cada vez más procesos de negocio.

Sin embargo, ese crecimiento dio lugar a una nueva necesidad. Los usuarios ya no esperaban únicamente obtener información: comenzaron a solicitar que el propio sistema realizara acciones, como validar un distribuidor, registrar un reclamo, consultar distintos sistemas corporativos o iniciar una operación comercial. En ese momento apareció un nuevo desafío: **¿cómo lograr que un asistente no solo informe, sino que además sea capaz de decidir qué hacer y ejecutar esas acciones?**

Imaginemos algunas situaciones cotidianas que comenzaron a presentarse en Ortelana Textil:

- Un distribuidor consulta si cumple los requisitos para operar con cuenta corriente.
- Otro informa que recibió un lote de telas con defectos y desea iniciar un reclamo.
- Un tercero solicita reservar la mercadería disponible antes de que otro cliente concrete la compra.

En todos estos casos, el sistema necesita interactuar con diferentes aplicaciones corporativas, consultar información en tiempo real e incluso ejecutar acciones sobre ellas. El problema ya no consiste únicamente en responder una pregunta en base a información actualizada. Ahora debe decidir qué pasos seguir, qué recursos utilizar y cómo combinarlos para completar una tarea. En otras palabras, ya no alcanza con recuperar conocimiento: es necesario razonar sobre el problema y actuar sobre el entorno.

A esta altura, es habitual que surja una pregunta como la siguiente:

"Pero... si el asistente ya entiende lo que le pido, conoce la información de la empresa y puede responder correctamente... ¿por qué no le agregamos una parte que cree el reclamo y listo?"

La idea parece razonable y, de hecho, sería una solución válida si todas las solicitudes fueran idénticas y siguieran siempre el mismo procedimiento. Sin embargo, las situaciones reales rara vez funcionan de esa manera. Antes de registrar un reclamo, el sistema podría necesitar verificar la identidad del distribuidor, consultar el estado de la

garantía, determinar el tipo de inconveniente informado, decidir qué área debe intervenir e incluso solicitar información adicional si los datos disponibles no son suficientes.

En otras palabras, el tema **ya no pasa por ejecutar** una rutina determinada, **sino por decidir** cuál es el proceso adecuado para cada situación.

Este tipo de problemas requiere sistemas capaces de **analizar una situación**, **decidir qué acciones realizar**, empleando las herramientas necesarias y **adaptando su comportamiento en función de los resultados obtenidos**. A este tipo de sistemas los denominamos **Agentes Inteligentes**.

1. El Salto Conceptual: Del Asistente al Agente

El asistente de Ortelana Textil es capaz de interpretar consultas expresadas en lenguaje natural, incluyendo jerga propia del negocio textil e incluso mensajes con errores de redacción u ortografía, recuperar información actualizada desde sus sistemas corporativos y generar respuestas precisas y contextualizadas. Gracias a las arquitecturas RAG y a las mejoras incorporadas en las últimas etapas del desarrollo, se convirtió en una herramienta confiable para asistir tanto a usuarios externos como internos en sus consultas diarias.

Sin embargo, todas esas capacidades comparten una característica en común: **el objetivo del sistema continúa siendo responder una consulta**. Una vez generada la respuesta, su tarea ha finalizado, aunque el usuario todavía deba realizar distintas acciones para resolver el problema planteado.

Un **Agente Inteligente** representa el siguiente paso en esta evolución. Además de comprender el problema y generar respuestas, es capaz de decidir qué acciones llevar a cabo para alcanzar un objetivo, empleando las herramientas necesarias y adaptando su comportamiento en función de los resultados obtenidos durante la ejecución. En lugar de limitarse a informar, participa activamente en la resolución de la tarea.

Para comprender mejor esta diferencia, imaginemos nuevamente la situación de Ortelana Textil:

Consulta	Asistente	Agente
"¿Cómo registro un reclamo?"	Explica el procedimiento.	Inicia el reclamo, solicita la información faltante e inicia el proceso de reclamo en los sistemas de la empresa.
"¿Hay stock?"	Informa el stock.	Consulta el stock y, si el usuario lo solicita, realiza la reserva.

"¿Puedo comprar a cuenta corriente?"	Explica las condiciones.	Verifica los requisitos del distribuidor y responde según el resultado.

En cierta forma, el asistente es pasivo: interpreta la consulta, recupera información, la procesa y genera una respuesta, pero su intervención termina allí. No modifica el estado de los sistemas con los que trabaja ni decide qué operación debe realizarse a continuación. Su función consiste fundamentalmente en **informar**.

Un agente inteligente, en cambio, es parte activa del circuito. A partir de un objetivo, puede decidir qué acciones realizar, utilizar las herramientas disponibles para llevarlas a cabo y evaluar los resultados obtenidos para determinar cómo continuar. De esta manera, el sistema deja de limitarse a producir información y pasa a **interactuar con el entorno en el que debe resolver la tarea**.

Esta diferencia modifica una característica fundamental de la arquitectura. En un flujo tradicional, los pasos que deben ejecutarse están definidos de antemano: una operación conduce a la siguiente siguiendo un recorrido establecido. En un agente, en cambio, el siguiente paso puede depender de lo que ocurrió en el anterior. **El flujo de ejecución depende de cada situación contemplada y se adapta a partir de las decisiones que deben tomarse tras cada paso.**

Un distribuidor informa que recibió un producto y desea devolverlo. El sistema debe identificar al distribuidor y la operación involucrada, verificar que la compra exista y consultar las condiciones comerciales aplicables. Si la devolución está contemplada, debe determinar si corresponde retirar la mercadería, reemplazarla o emitir una nota de crédito. Para tomar esa decisión puede necesitar consultar el estado del pedido, el motivo informado por el distribuidor y la disponibilidad de una unidad de reemplazo.

El resultado de cada consulta puede modificar el camino a seguir. Si el pedido no existe, deberá informar el inconveniente. Si la devolución no está contemplada para ese caso, deberá comunicar las condiciones correspondientes. Si procede un reemplazo pero no hay disponibilidad, deberá considerar otra alternativa. Solo cuando se cumplan las condiciones necesarias podrá avanzar con la acción correspondiente.

Pero supongamos que el distribuidor informa que quiere devolverlo porque recibió un producto equivocado. Si bien parte del circuito es el mismo, seguramente necesitará otros controles y otras decisiones. Por ejemplo, **deberá verificar si esa entrega equivocada no involucra a otro cliente**.

En procesos donde las decisiones **dependen de la situación concreta**, **no existe una secuencia única de pasos** que pueda ejecutarse de principio a fin para cualquier solicitud. El sistema debe **decidir qué hacer a continuación** a partir de la información que va obteniendo.

Actuadores (Actuators)

Para poder actuar sobre su entorno, un agente necesita disponer de recursos que le permitan consultar información o ejecutar operaciones en los sistemas con los que interactúa. Estos recursos pueden ser muy diversos: una consulta a una base de datos, una API de un sistema corporativo, una función que realiza un cálculo o incluso una operación que modifica información.

A estos recursos los denominamos **herramientas (tools)**. Una herramienta encapsula una capacidad concreta que el agente puede utilizar cuando la necesita. El agente no tiene por qué conocer cómo está implementada internamente; debe conocer qué puede hacer con ella, qué información necesita para utilizarla y qué resultado puede obtener.

Esto introduce una separación importante: **el agente decide qué necesita hacer; la herramienta se encarga de realizar la operación concreta**.

Inversión de Control de Flujo

El cambio más drástico ante esta situación no es el "poder" del modelo, sino quién tiene el control de la ejecución. En las arquitecturas **RAG tradicionales**, el programador define un pipeline rígido: la consulta entra, se vectoriza obligatoriamente, se recuperan fragmentos y se genera una respuesta. En este esquema, el código dicta el camino y el LLM es un analizador de texto en nodos fijos.

En un **Agente Inteligente**, el programador ya no define una secuencia predeterminada (*interpretar, recuperar información, responder*), sino que establece un **entorno con capacidades (herramientas)** y un **objetivo**. Es el modelo quien, en tiempo de ejecución, determina qué acción resulta adecuada a partir del objetivo, del estado actual y de los resultados obtenidos. El orquestador se encarga de ejecutar esa decisión y devolver el resultado al contexto del agente.

En términos simples: **hasta ahora, el flujo determinaba lo que pasaba; a partir de ahora, lo que pasa determina el flujo.**

Bajo este paradigma, las acciones no responden a un guión rígido, sino que **reaccionan a cada estado del sistema**. El agente evalúa continuamente la situación y elige dinámicamente el mejor camino para alcanzar el objetivo planteado, adaptando su recorrido paso a paso según las circunstancias que se presentan.

2. El Ciclo de Vida del Agente (Patrón ReAct & Marco PEAS)

Mientras que una cadena **RAG sigue un recorrido definido** por la arquitectura —recuperación, procesamiento del contexto y generación—, **un agente es intrínsecamente cíclico**: evalúa la situación, ejecuta acciones y vuelve a evaluar los

resultados obtenidos hasta satisfacer una condición de corte (como resolver el objetivo, solicitar aclaraciones al usuario o detectar un límite de ejecución).

Para materializar este comportamiento dinámico, la arquitectura de los agentes modernos adopta el patrón **ReAct (Reasoning + Acting)**. Este enfoque combina la capacidad del modelo para evaluar la situación y seleccionar acciones con la posibilidad de ejecutarlas sobre el entorno, estructurando cada iteración del bucle en tres fases fundamentales:

- **Pensamiento (*Thought*):** El modelo analiza el estado actual, el objetivo asignado y el historial acumulado de la conversación. En esta etapa razona sobre qué información necesita, detecta inconsistencias y decide qué paso intermedio dar a continuación.
- **Acción (*Action*):** Cuando el modelo determina que requiere interactuar con el entorno, emite una instrucción estructurada destinada a invocar una herramienta externa específica con parámetros concretos (por ejemplo, consultar el stock de un artículo o verificar el saldo de una cuenta corriente).
- **Observación (*Observation*):** El sistema ejecuta la herramienta solicitada y devuelve el resultado fáctico al contexto del modelo. Esta nueva información pasa a formar parte del estado, permitiendo que el agente evalúe si la acción fue exitosa o si debe ajustar su estrategia en el siguiente ciclo de razonamiento.

Este ciclo de **Pensar** → **Actuar** → **Observar** se repite de manera iterativa, y la acción en cada ciclo dependerá de cómo ha reaccionado el entorno y el estado acumulado hasta ese momento. El proceso no concluye hasta que el modelo determina que cuenta con la información suficiente para resolver la tarea encomendada, generar la respuesta final o hasta que se alcanza una condición de parada establecida por la arquitectura.

El Patrón ReAct en Acción

Para comprender cómo opera este ciclo en la práctica, veamos cómo se configura el entorno operativo del agente en **Ortelana Textil**.

El desarrollador no programa un flujo rígido paso a paso (no escribe condiciones anidadas que obliguen a validar crédito antes de consultar stock). En su lugar, simplemente habilita un **conjunto de herramientas (*tooling*)** y define las reglas y contratos de datos para utilizarlas:

- consultar_cuenta_corriente(cuit: str): Consulta el estado comercial y el margen crediticio disponible del distribuidor.
- consultar_producto(nombre: str): Busca un producto por su nombre y recupera su id.
- consultar_stock(articulo_id: str, metros: float): Verifica la disponibilidad física en depósito.

- obtener_cotizacion(articulo_id: str, metros: float): Calcula el costo total según la lista de precios vigente y convenios.
- afectar_credito_cliente(cuit: str, monto: float): Realiza la mutación financiera restando el importe del saldo utilizable del cliente.
- crear_reserva(cuit: str, articulo_id: str, metros: float): Bloquea el stock en el ERP e imputa el compromiso financiero en la cuenta corriente.

El conjunto de herramientas puede expandirse para incluir diversas capacidades operativas, como la cancelación de reservas, la búsqueda de alternativas textiles ante falta de stock o el inicio de procesos de reclamo y devoluciones. No obstante, es fundamental comprender que el valor arquitectónico del agente no reside en la ejecución de una rutina, sino en su capacidad para **orquestar sistemas desacoplados** (finanzas, inventario, ventas).

En la ingeniería de software tradicional, a menudo se busca simplificar la interfaz mediante "super-funciones". Sin embargo, en el diseño de sistemas inteligentes, el poseer **herramientas funcionalmente mínimas** y enfocadas en una **única responsabilidad** permite que el conjunto sea mucho más flexible, eficiente y escalable. Este enfoque de responsabilidad única permite que el agente actúe como un verdadero **director de orquesta** sobre procesos de negocio reales de Ortelana Textil.

Al fragmentar las capacidades, hacemos que **cada operación** constituya una **decisión independiente dentro del flujo**. Esto garantiza que éste no sea una "caja negra" de código rígido, sino un **circuito auditable** donde cada decisión se fundamenta en la observación directa del estado actual del entorno. Ya expuesto este entorno de capacidades y definido el objetivo, se produce la **inversión de control de flujo**: el agente deja de seguir un recorrido predeterminado y pasa a decidir, paso a paso, el camino hacia la resolución del objetivo de forma flexible y basándose en los estados que va observando.

Traza de Ejecución de Ejemplo

Veamos un ejemplo concreto a partir del mensaje recibido por Ortelana Textil de un cliente que quiere hacer una reserva, condicionándola a su situación crediticia:

> *"Hola, soy Distribuidora Norte (CUIT 30-71234567-8). Necesitamos reservar 300 metros de Gabardina Pesada Azul Marino si tenemos margen disponible en nuestra cuenta corriente."*

A partir del mensaje recibido, el agente analiza la solicitud y, **con lo que tiene a mano** en su caja de herramientas, **construye dinámicamente** el camino necesario para resolver la necesidad**, examinando la situación** tras cada acción y **decidiendo** el siguiente paso **en tiempo real**.

Para seguir la ejecución del agente desde el mensaje recibido hasta la respuesta, a partir de las herramientas declaradas y en función al pedido y a datos simulados,

utilizaremos las categorías **Pensamiento**, **Acción** y **Observación** para describir cada paso del ciclo ReAct:

```
None
[Iteración 1]

Pensamiento: El usuario solicita una reserva condicionada a su margen de
crédito. Antes de verificar el stock físico, debo corroborar el estado
comercial y el límite disponible del CUIT 30-71234567-8.

Acción: consultar_cuenta_corriente(cuit="30-71234567-8")

Observación: {"estado": "Habilitado", "margen_disponible": 1500000,
"moneda": "ARS"}

[Iteración 2]

Pensamiento: El distribuidor está habilitado con un margen de $1.500.000.
El siguiente paso es verificar si existe un producto que matchee con
"Gabardina Pesada Azul Marino" y conseguir su identificador técnico.

Acción: consultar_producto("Gabardina Pesada Azul Marino")

Observación: {"id": "TEL-00427", "descripcion": "Gabardina 12 OZ Azul
Marino"}

[Iteración 3]

Pensamiento: El sistema identificó que la gabardina pesada corresponde al
artículo TEL-00427 ("Gabardina 12 OZ Azul Marino"). Ahora debo confirmar
si hay 300 metros disponibles de esta partida técnica específica.

Acción: consultar_stock(articulo_id="TEL-00427", metros=300)

Observación: {"disponible": true, "sucursal": "Córdoba"}

[Iteración 4]

Pensamiento: Hay stock suficiente, pero para validar contra el crédito
necesito el costo total exacto antes de afectar el saldo.

Acción: obtener_cotizacion(articulo_id="TEL-00427", metros=300)

Observación: {"total_operacion": 960000, "moneda": "ARS"}

[Iteración 5]

Pensamiento: El total ($960.000) es menor al margen ($1.500.000). Procedo
a comprometer financieramente el saldo del cliente.
```

```
Acción: afectar_credito_cliente(cuit="30-71234567-8", monto=960000)

Observación: {"estado": "Saldo afectado", "nuevo_disponible": 540000}

[Iteración 6]

Pensamiento: Saldo afectado con éxito. Finalmente, ejecuto el bloqueo
físico de la tela.

Acción: crear_reserva(cuit="30-71234567-8", articulo_id="TEL-00427",
metros=300)

Observación: {"id_reserva": "RES-89421", "estado": "Confirmada"}

[Iteración 7 - Condición de Corte]

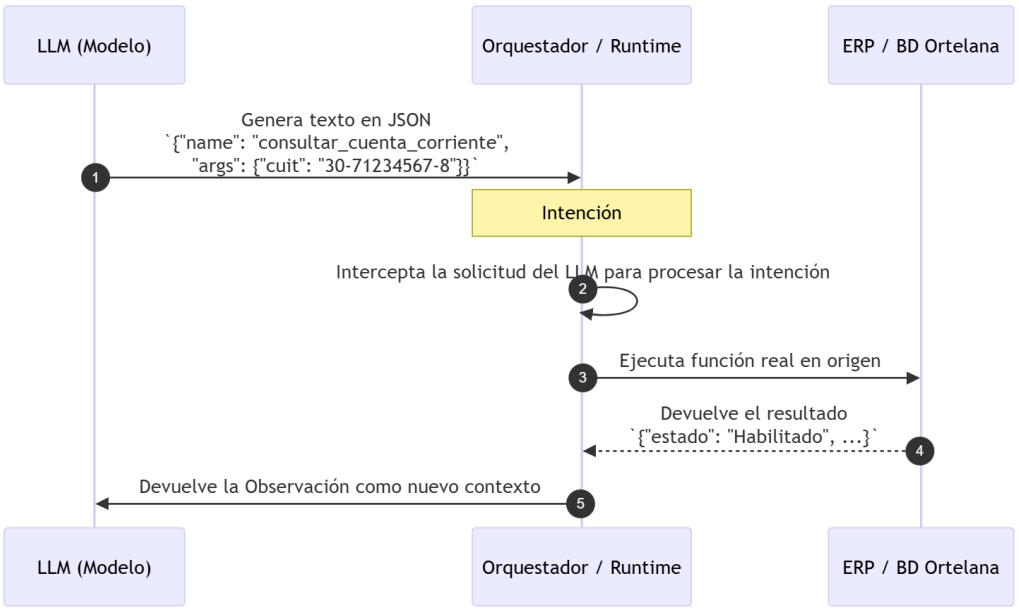
Pensamiento: La reserva fue generada con éxito con el identificador
RES-89421 y se verificaron todas las condiciones de negocio. Dispongo de
toda la información para responder al distribuidor.

Respuesta Final: "Estimado cliente de Distribuidora Norte, confirmamos la
reserva de 300 metros de Gabardina Azul Marino de 12 onzas bajo el
comprobante RES-89421. El monto de la operación ($960.000) fue afectado a
su cuenta corriente, quedándole un saldo disponible de $540.000. La
mercadería permanecerá reservada por 72 horas hábiles."
```

3. La Trastienda Técnica: Desmitificando el *Pensamiento* del Agente

Al analizar una traza de ejecución como la de Ortelana Textil, es muy común **humanizar el comportamiento del modelo de lenguaje**: pareciera que el modelo “está ejecutando código”, “conectándose a una base de datos SQL” o “interactuando directamente con el sistema de inventario”.

Nada de eso lo hizo directamente el modelo. **El LLM es un procesador de texto probabilístico; no ejecuta código, no realiza peticiones HTTP ni modifica bases de datos.** Para que el sistema pueda interactuar con el mundo exterior se requiere un mecanismo de traducción entre las intenciones expresadas por el modelo y las capacidades ejecutables del sistema, denominado **Function Calling**. El diagrama a continuación explica, de forma simplificada, los pasos de esta interacción:



Cada paso del ciclo **Acción** → **Observación** responde a la siguiente división estricta de responsabilidades:

1. **Emisión de la intención (Acción):** En la Iteración 1, cuando el modelo "decide" consultar el margen crediticio, produce una **solicitud estructurada de invocación de una herramienta**. Esta solicitud expresa la intención de utilizar una función específica con argumentos concretos: `{"name": "consultar_cuenta_corriente", "args": {"cuit": "30-71234567-8"}}`.
2. **Intercepción por el Runtime:** El orquestador de la aplicación (el entorno en Python que envuelve la llamada al LLM) **recibe e interpreta esta solicitud**. Identifica la herramienta solicitada, valida los datos recibidos como parámetros y **ejecuta el código Python real** en el servidor de la empresa.
3. **Ejecución Real:** El código Python efectúa la llamada API al ERP de Ortelana o consulta la base de datos SQL.
4. **Reinyección de la Observación:** El orquestador toma la respuesta de la base de datos, la transforma en un mensaje de *Observación* y la agrega a la ventana de contexto de la conversación.
5. **Re-evaluación del LLM:** Se realiza una nueva llamada al LLM enviándole todo el historial acumulado (incluyendo el resultado que acaba de obtener). Con esa información, el modelo genera el siguiente "Pensamiento".

El Docstring y la Firma como Prompt de Invocación

Dado que el código fuente interno de la función Python es completamente invisible para el modelo, **¿cómo aprende el agente qué actuador elegir y cómo pasarle parámetros?**

Si el LLM no puede ver el código interno de la función Python (o lo que sea que debe ejecutar), **¿cómo supo que consultar_cuenta_corriente requería un parámetro cuit en formato de texto y no el nombre de la empresa?**

La respuesta reside en que **el modelo recibe, al iniciar la llamada, una descripción estructurada de los actuadores existentes, construida a partir de sus nombres, firmas y documentación textual (docstring)**. Esta definición se mantiene disponible durante la misma.
El desarrollador no le escribe código al modelo: le redacta especificaciones descriptivas:

- **Descripción/documentación** → ayuda al modelo a elegir la herramienta y construir los argumentos.
- **Esquema de tipos/validación** → permite al runtime validar estructuralmente esos argumentos antes de ejecutar la función.

```python
# La firma y el docstring le enseñan al modelo CÓMO y CUÁNDO usar la herramienta
@tool
def consultar_cuenta_corriente(cuit: str) -> dict:
    """
    Consulta el estado comercial y el margen crediticio disponible de un distribuidor registrado.
    Utilizar ESTA función únicamente cuando se necesite verificar si el cliente tiene saldo suficiente
    para realizar una compra o reserva a cuenta corriente.

    Args:
        cuit: CUIT del distribuidor en formato String de 11 dígitos sin guiones. Ejemplo: '30712345678'
    """
    ...
```

Sí debemos tener en cuenta estas dos cosas:

- **Riesgo de ambigüedad:** Si la descripción provista en el *docstring* fuera vaga o ambigua, el agente podría cometer errores de selección o de invocación a actuadores, o inventar parámetros inexistentes, lo que se suele relacionar con las **alucinaciones de** (*tool hallucination*).
- **Validación de tipos:** Una descripción explícita, precisa y estructurada mediante esquemas Pydantic que permite al runtime validar la sintaxis de los parámetros generados por el LLM antes de enviarlos al backend corporativo, evitando errores de ejecución en la base de datos.

Una cuestión importante a remarcar es que la estructuración de una solución como la que presentamos, establecida en capas lógicas, con el desacoplamiento estricto entre la intención (probabilística, emitida por el modelo) y la ejecución (determinista, que corre en nuestro servidor) nos otorga un beneficio de diseño fundamental: la independencia del proveedor del LLM. Como el modelo de lenguaje solo se limita a procesar texto y devolver estructuras, podemos alternar de forma transparente entre un modelo comercial en la nube o un modelo local de código abierto (como Ollama) si el negocio exige procesar datos impositivos o financieros bajo políticas estrictas de privacidad.

La Arquitectura Detrás del "Pensamiento" del Agente

A esta altura podemos describir el comportamiento del agente diciendo que **analiza la situación, decide qué hacer, utiliza una herramienta y vuelve a evaluar el resultado**. Esta forma de explicarlo resulta útil para comprender el ciclo ReAct, pero contiene una simplificación que es importante hacer explícita.

Cuando decimos que *"el agente piensa y decide"*, no significa que el modelo de lenguaje ejecute directamente las acciones que observamos en el sistema. Tampoco significa que exista dentro del modelo una entidad autónoma que pueda consultar una base de datos, enviar una solicitud HTTP o modificar el ERP de la compañía.

La expresión describe el **comportamiento observable de una arquitectura compuesta por varios componentes**, en la que cada uno tiene una responsabilidad concreta.

Una metáfora útil, pero que tiene límites

En la explicación del ciclo ReAct utilizamos expresiones como:

> **Pensamiento:** "Necesito verificar el margen disponible antes de consultar el stock."
>
> **Acción:** `consultar_cuenta_corriente(...)`
>
> **Observación:** `{"estado": "Habilitado", "margen_disponible": 1500000}`

Esta representación resulta cómoda para seguir el *razonamiento* del agente. Sin embargo, desde el punto de vista arquitectónico, lo que ocurre es diferente.

El modelo recibe el estado disponible y, a partir de la situación planteada, **determina si necesita utilizar una herramienta y cuál de las disponibles resulta adecuada para continuar**. Cuando identifica una herramienta que puede satisfacer la necesidad detectada, produce una **salida estructurada con la intención de invocarla, junto a los argumentos necesarios**. Esa salida es interpretada por el sistema de ejecución,

que identifica la herramienta solicitada, valida sus parámetros y ejecuta el código correspondiente, cuyo resultado de ejecución alimenta al contexto del modelo.

Podemos resumir ambas perspectivas:

Explicación didáctica	Arquitectura real
El agente **piensa.**	El modelo procesa el estado y genera una salida.
El agente **decide usar una herramienta.**	El modelo produce una solicitud estructurada de invocación.
El agente **ejecuta la acción.**	El runtime interpreta la solicitud y ejecuta la herramienta.
El agente **observa el resultado.**	El resultado de la ejecución se incorpora nuevamente al estado.
El agente **decide cómo continuar.**	El modelo vuelve a evaluar el estado actualizado.

La metáfora no es incorrecta: describe adecuadamente el comportamiento del sistema desde una perspectiva funcional. Lo importante aquí es **no confundir ese comportamiento con la implementación que lo hace posible**.

El modelo de lenguaje no conoce por sí mismo cómo consultar el ERP ni ejecuta el código de `consultar_stock`. Su capacidad de actuar depende del conjunto de capacidades que la arquitectura pone a su disposición y de los mecanismos que controlan su ejecución.

¿Qué puede hacer realmente un agente?

Esta pregunta nos lleva a una cuestión fundamental del diseño de sistemas agénticos.

Supongamos que Ortelana Textil pone a disposición del agente las siguientes herramientas:

```
None
consultar_stock(...)
consultar_cuenta_corriente(...)
obtener_cotizacion(...)
crear_reserva(...)
```

El agente puede decidir utilizar esas capacidades, combinarlas y determinar en qué orden necesita hacerlo. Pero no puede decidir utilizar una capacidad que **no existe en su entorno**.

Si no dispone de una herramienta para cancelar una reserva, no puede cancelar una reserva simplemente porque el modelo "sepa" cómo debería hacerse.

Esto significa que la autonomía del agente está acotada por la arquitectura que lo rodea.

Podemos expresarlo de una manera sencilla:

> **El modelo puede decidir qué hacer dentro del espacio de acciones que el sistema le permite realizar.**

Por eso, en un sistema agéntico, diseñar correctamente las herramientas es tan importante como elegir el modelo de lenguaje.

El desafío del diseñador: diseñar las capacidades

Una herramienta no es simplemente una función que el programador pone a disposición del modelo. Es una **capacidad operativa que se incorpora al espacio de decisiones del agente**.

Lo primero que hay que entender es que **el modelo trabaja con texto**. Aunque nosotros representemos una herramienta como una función de Python, para el modelo esa función no existe como código ejecutable. Lo que recibe es una **descripción textual de una capacidad**, junto con la información necesaria para solicitar su ejecución y comprender el resultado.

Por eso, el nombre de la herramienta, la descripción de lo que hace, sus parámetros y el significado de sus valores de retorno no son detalles secundarios de implementación: **constituyen el lenguaje mediante el cual la arquitectura le comunica al modelo qué puede hacer y qué ocurrió cuando lo hizo.**

El nombre de la herramienta debe expresar con claridad la capacidad que ofrece. Sus parámetros deben indicar qué información necesita y qué representa cada valor. Del mismo modo, el resultado que devuelve debe tener un significado inequívoco para que el agente pueda utilizarlo como evidencia para decidir el siguiente paso.

No es lo mismo una herramienta llamada `consultar_stock()` que una llamada `gestionar_producto()`. La primera expone una capacidad concreta; la segunda obliga al modelo a inferir qué operación puede realizar y en qué circunstancias.

Lo mismo ocurre con los parámetros. Un argumento como `cantidad=300` carece de significado suficiente si el sistema no deja claro si representa **metros, unidades,**

kilogramos o pesos. Incluso cuando el nombre del parámetro es correcto, sus valores deben conservar un significado preciso y consistente.

Finalmente, el valor de retorno forma parte del contrato de la herramienta. Una respuesta como `True` puede indicar que existe stock, que la operación fue exitosa o simplemente que la consulta pudo ejecutarse. En cambio, un resultado estructurado como `{"disponible": true, "cantidad": 450, "unidad": "metros"}` proporciona al agente información explícita sobre la situación que acaba de observar.

En un sistema agéntico, estos detalles no son solamente cuestiones de programación. **Constituyen parte de la información que utiliza el modelo para tomar sus siguientes decisiones.**

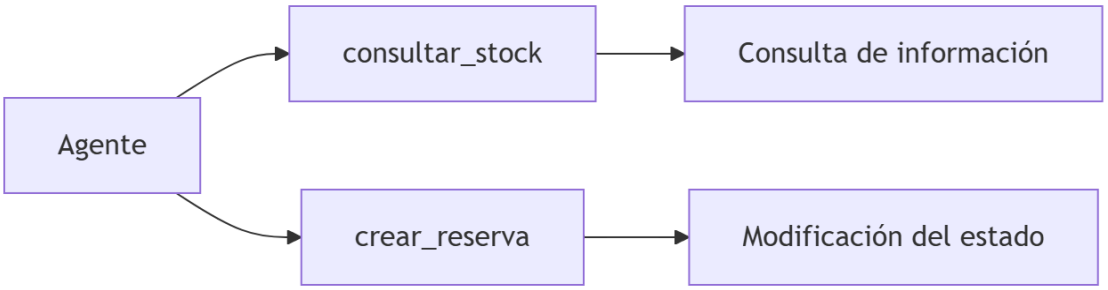
Consideremos dos herramientas declaradas previamente para Ortelana Textil:

```
None
consultar_stock(articulo_id, metros)
crear_reserva(cuit, articulo_id, metros)
```

Aunque ambas estén relacionadas con la disponibilidad de mercadería y se programen con la misma sintaxis en Python (utilizando el decorador `@tool`), tienen responsabilidades y niveles de impacto completamente diferentes sobre el entorno:

1. **Herramientas de Percepción (Sensores)**: `consultar_stock` actúa esencialmente como un sensor enriquecido. Su función es leer el entorno y traer evidencia factual (¿cuánta tela hay en Córdoba?) sin producir ningún efecto secundario. La consulta de información es segura y de bajo riesgo operativo.
2. **Herramientas de Acción (Actuadores)**: `crear_reserva`, en cambio, es un actuador puro. Su ejecución muta el estado del mundo real: bloquea mercadería física en el inventario, genera un compromiso comercial firme y altera los saldos financieros en el ERP.

La diferencia es fundamental:

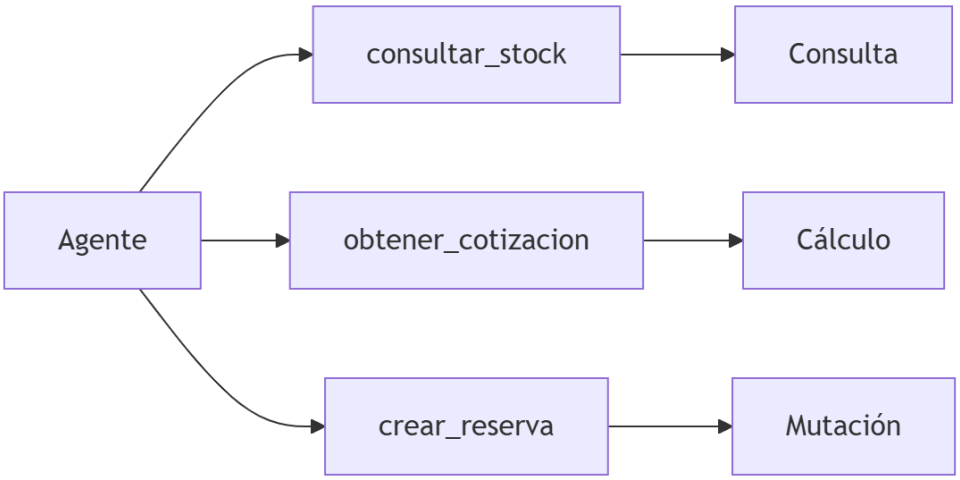


Ambas capacidades aparecen ante el modelo como herramientas disponibles, pero no tienen el mismo nivel de impacto sobre el entorno.

Por esta razón, un buen actuador debe tener una **responsabilidad clara** y un **contrato extremadamente preciso**: qué hace, qué datos necesita, qué devuelve y qué efectos secundarios puede producir. Esto recupera un principio clave: **delimitar de forma asimétrica las herramientas de lectura (sensores) y las de escritura (actuadores) facilita que el agente seleccione correctamente la acción que necesita y nos permite blindar el backend ante ejecuciones erróneas**.

Cuanto más ambiguas sean las herramientas, mayor será el espacio de decisiones que el modelo deberá distinguir. Por el contrario, cuando las capacidades están bien separadas, el agente dispone de piezas pequeñas que puede combinar según la situación. No se trata entonces de construir una herramienta que haga “todo lo necesario para gestionar una reserva”, sino de exponer capacidades que representen responsabilidades concretas.

La arquitectura puede quedar así:



El agente decide **qué capacidad necesita**. Cada herramienta se ocupa de **hacer aquello para lo que fue diseñada**.

Esta separación permite además auditar el comportamiento del sistema: podemos observar qué herramienta fue seleccionada, con qué parámetros y qué resultado produjo.

El RAG como sensor de conocimiento

Esta distinción entre sensores y actuadores nos permite redefinir y elevar un concepto central presentado previamente: **RAG**. Con RAG, el pipeline de recuperación era el sistema completo. En la arquitectura agéntica, en cambio, RAG deja de ser el

contenedor para convertirse en una herramienta de percepción (un sensor enriquecido) más dentro del catálogo del agente.

Cuando el agente detecta que la consulta requiere validar una normativa de crédito, una política de devolución o una ficha de producto, no inicia un flujo informativo aislado; simplemente invoca de manera autónoma su herramienta de RAG como un sensor para recuperar la información que necesita para continuar con su ciclo de razonamiento.

Delegando responsabilidades: especialización de tareas

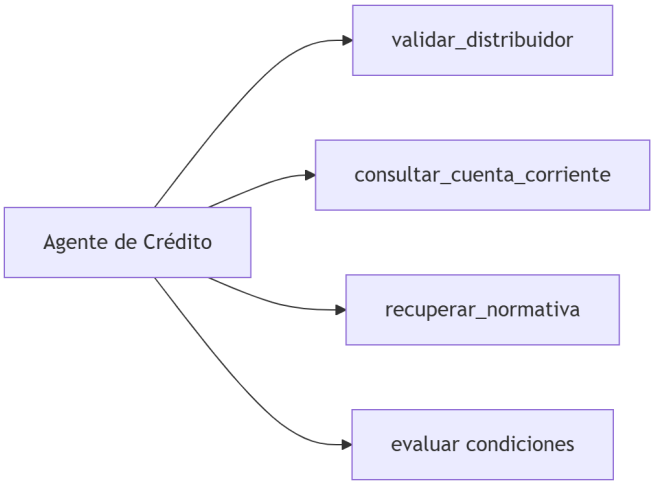
Llegados a este punto aparece una pregunta interesante:

> ***¿Puede un agente delegar trabajo en otro agente?***

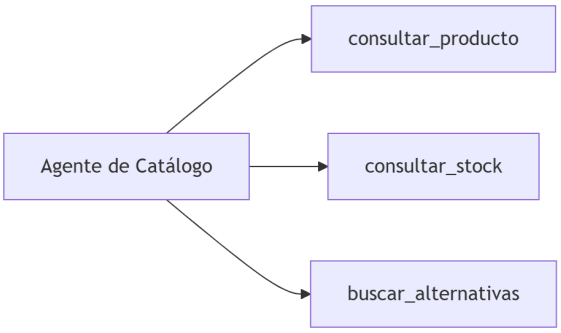
Claro que sí.

Desde el punto de vista del agente que lo invoca, otro agente puede presentarse como una **capacidad disponible**, del mismo modo que cualquier otro actuador. La diferencia es que mientras un **actuador simple** (como `consultar_stock`) encapsula una operación de software directa, mientras que un **subagente delegado** (como el *Agente de Crédito*) **actúa ante el orquestador como un actuador abstracto que encapsula un proceso completo de toma de decisiones dentro de un dominio determinado**. El orquestador solo ve una capacidad de alto nivel ("Resolver situación crediticia") y se abstrae de la complejidad interna del subdominio.

Por ejemplo, podríamos tener un agente especializado en crédito:

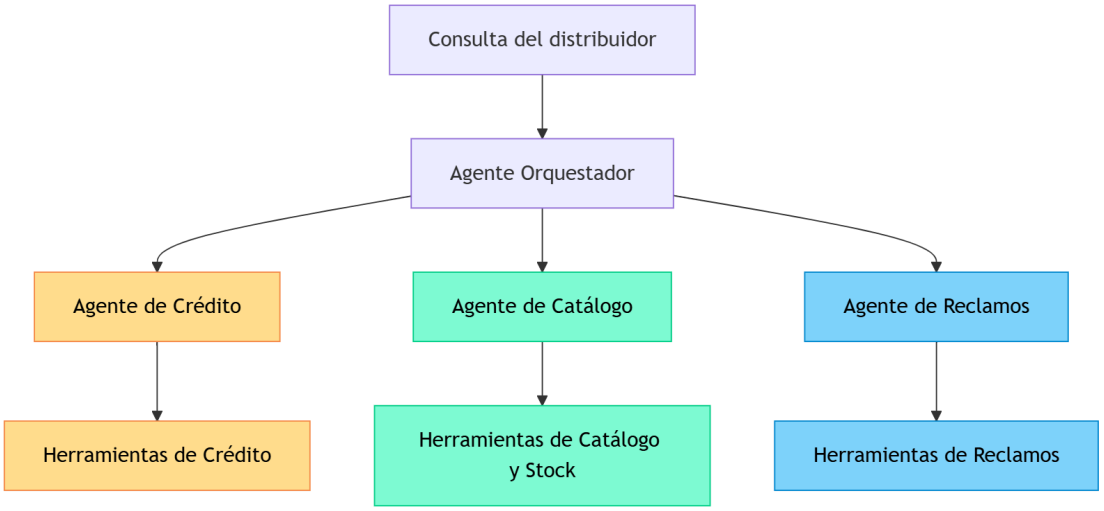


Otro agente podría encargarse de catálogo y stock:



Para un sistema que necesita resolver una solicitud comercial completa, no resulta necesario que un único agente conozca en detalle todas esas capacidades.

Podemos incorporar un **agente orquestador** que determine a quién corresponde cada parte de la tarea:



Aquí aparece una propiedad arquitectónica especialmente importante: **la delegación permite abstraer la complejidad del dominio delegado**.

El orquestador no necesita conocer todos los pasos internos necesarios para resolver una cuestión crediticia. Puede limitarse a determinar:

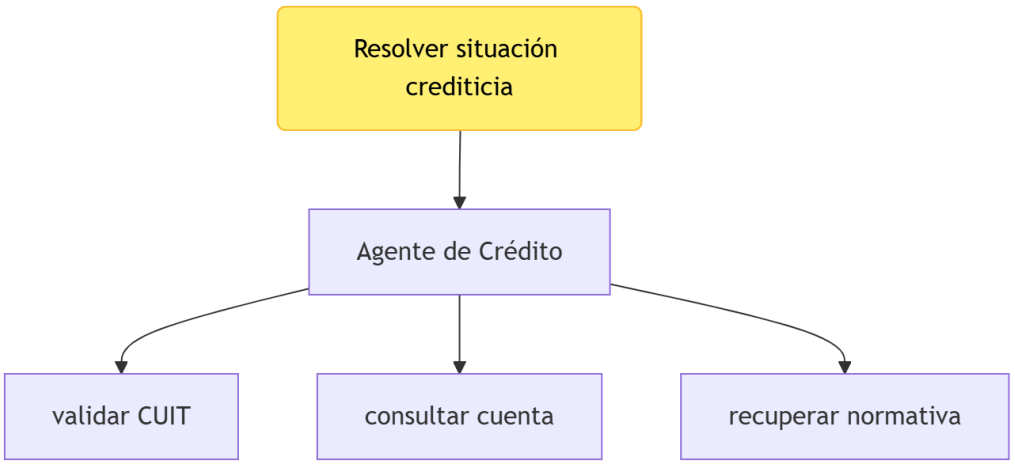
> "Esta parte de la solicitud corresponde al dominio de Crédito."

A partir de allí, el agente especializado se ocupa de analizar la situación, seleccionar sus propias herramientas y completar la tarea dentro de su ámbito.

El orquestador trabaja entonces con una capacidad de nivel superior:

```
None
Resolver situación crediticia
```

mientras que el agente especializado se ocupa de la complejidad necesaria para hacerlo:



La complejidad no desapareció. **Fue encapsulada detrás de una responsabilidad bien definida.**

Esta es una de las razones por las que una arquitectura multiagente puede resultar conveniente: no necesariamente porque “varios agentes sean más inteligentes que uno”, sino porque permite **distribuir responsabilidades y mantener abstraída la complejidad de cada dominio**.

El agente orquestador no necesita saber *cómo* se resuelve una cuestión crediticia, de la misma manera que un agente de crédito no necesita conocer los detalles internos del sistema de reclamos.

Cada componente puede concentrarse en aquello que pertenece a su responsabilidad.

De la autonomía aparente a la responsabilidad arquitectónica

La discusión anterior permite revisar una idea que puede resultar engañosa cuando hablamos de agentes:

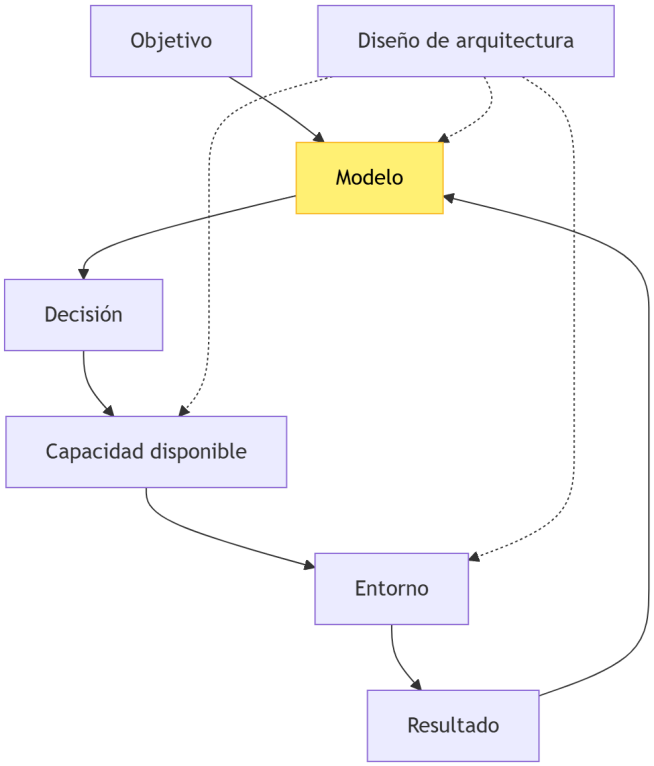
Cuanto más autónomo parece un sistema, más importante se vuelve el diseño de aquello que **queda fuera del modelo**.

El modelo puede seleccionar una herramienta, pero alguien tuvo que decidir que esa herramienta existiera. Puede elegir entre varias capacidades, pero alguien tuvo que definir cuáles son esas capacidades y bajo qué condiciones pueden utilizarse. Puede

delegar una tarea a otro agente, pero alguien tuvo que establecer qué responsabilidad tiene ese agente y qué herramientas puede utilizar. Puede continuar un proceso a partir de los resultados obtenidos, pero la arquitectura debe establecer cómo se conserva el estado, cómo se controla el ciclo y cuándo debe detenerse.

Por eso, construir un agente no consiste simplemente en conectar un modelo de lenguaje con algunas funciones, implica diseñar un **entorno operativo** en el que el modelo pueda tomar decisiones, pero dentro de un conjunto de capacidades, estados y reglas que han sido cuidadosamente definidos.

Podemos representarlo de la siguiente manera:



El modelo ocupa un lugar central en el ciclo, pero **no constituye por sí solo el agente completo**.

El comportamiento que observamos emerge de la interacción entre el modelo, el estado, las herramientas, los mecanismos de ejecución y las reglas que organizan ese conjunto.

Y esto nos devuelve a la idea de inversión de control de flujo que introdujimos anteriormente: el sistema ya no sigue simplemente una secuencia escrita de antemano, pero tampoco funciona sin estructura.

El desafío consiste precisamente en construir una arquitectura que permita **delegar decisiones sin perder control sobre su ejecución**.

Hacia una arquitectura explícita

Hasta ahora describimos el ciclo ReAct principalmente como una secuencia conceptual:

```
None
Pensamiento → Acción → Observación → Pensamiento → ...
```

Ahora podemos observar que, para implementarlo de manera robusta, necesitamos representar explícitamente:

- qué información constituye el **estado**;
- qué componentes pueden **procesarlo**;
- qué capacidades pueden **ejecutarse**;
- cómo se determina el **siguiente paso**;
- cómo se conserva la información obtenida;
- y bajo qué condiciones termina el ciclo.

Cuando el sistema incorpora además varios agentes especializados, estas necesidades se vuelven todavía más evidentes.

Ya no alcanza con pensar en una sucesión de llamadas. Necesitamos una arquitectura capaz de representar **nodos de procesamiento, decisiones de routing, estado compartido y ciclos de ejecución**.

Volviendo a PEAS: Una Mirada Teórica sobre el Agente

Comprendida la mecánica interna de las herramientas, veremos que el entorno del agente se apoya en el **marco PEAS** (*Performance, Environment, Actuators, Sensors*):

- **Medida de Desempeño (*Performance*):** Resolver las solicitudes operativas de los clientes (como verificar crédito o reservar stock) de manera precisa, segura y minimizando la fricción, garantizando el cumplimiento de las reglas de negocio sin comprometer inventario ni crédito indebidamente.
- **Entorno (*Environment*):** El ecosistema digital corporativo, conformado por la base de datos de productos, el sistema ERP de inventario, el módulo comercial de cuentas corrientes y el canal de mensajería desde donde interactúa el distribuidor.
- **Actuadores (*Actuators*):** La batería de **Herramientas (*Tools*)** expuestas mediante *Function Calling* (crear_reserva, afectar_credito_cliente, consultar_stock, etc.). Son los únicos mecanismos mediante los cuales el agente puede modificar el estado del mundo real. Estos, por supuesto, pueden basarse en la inclusión de todas las herramientas que vimos previamente, por ejemplo Pydantic.

- **Sensores (*Sensors*):** El canal de entrada de mensajes en lenguaje natural enviados por los usuarios y los mensajes de **Observación** (datos estructurados JSON) que el runtime devuelve tras la ejecución de cada herramienta.

Este seguimiento de la ejecución, donde se puede observar cada decisión y acción del agente, demuestra que, en su arquitectura, cada paso es el resultado de un razonamiento dinámico basado tanto en el objetivo final como en la información capturada en tiempo real. El resultado de cada herramienta deja de ser un dato aislado para integrarse al contexto del modelo; esto le permite evaluar si debe persistir en su plan original, **pivotar hacia un curso de acción alternativo** o finalizar la tarea al haber satisfecho las condiciones de éxito.

Por ejemplo, ante una falta de stock o un crédito insuficiente, el flujo no se habría 'detenido' por un error de software; el agente habría razonado una salida diferente, desde informar la restricción comercial hasta ofrecer materiales o costos alternativos. Sin embargo, es fundamental recordar que **la autonomía del agente está rígidamente acotada por sus actuadores**: su inteligencia reside en la orquestación, pero su capacidad de resolución depende exclusivamente de la batería de herramientas disponible. **Sin las herramientas adecuadas para gestionar excepciones**, **la única respuesta honesta y fundamentada** del sistema ante un obstáculo **sería una negativa**.

Tools y diseño de software

Sabiendo ya que **el alcance operativo de un agente está condicionado por las capacidades que el sistema pone a disposición del LLM**, es importante entender que la IA no se programa a sí misma para incorporar nuevas capacidades al sistema. El agente solo puede percibir información y actuar sobre el entorno a través de los mecanismos que nosotros diseñamos y ponemos a su disposición.

Por eso, construir un sistema agéntico también requiere el trabajo de quien diseña el software: identificar qué capacidades necesita el agente para resolver las situaciones del dominio y construir las herramientas (**Tools**) que le permitan acceder a esa información o producir cambios en el entorno. En este sentido, podemos pensar algunas Tools como **sensores**, cuando permiten al agente conocer el estado del sistema, y otras como **actuadores**, cuando le permiten intervenir sobre él.

Tampoco significa que debamos dejar de lado los conocimientos adquiridos en el desarrollo de software tradicional. El surgimiento de los agentes inteligentes no implica empezar de cero en materia de diseño de software. No invalida principios como SOLID ni las técnicas y patrones utilizados en sistemas convencionales. Lo que cambia es el contexto en el que se aplican y, en algunos casos, la función que cumplen dentro de la arquitectura.

La **Inyección de Dependencias**, por ejemplo, sigue siendo útil al construir una

Tool. Una herramienta puede necesitar acceder a servicios de negocio, repositorios o clientes de APIs sin quedar acoplada a sus implementaciones concretas. Esto no solo favorece la **flexibilidad** del diseño, sino también su **extensibilidad**: a medida que el sistema incorpore nuevas funciones, podrán aparecer nuevos sensores y actuadores sin necesidad de modificar las herramientas existentes ni acoplarlas directamente a sus implementaciones.

La capacidad del agente para decidir **cuándo y para qué utilizar esas herramientas** se incorpora sobre esa arquitectura; no la reemplaza. Las Tools no deben entenderse simplemente como funciones que se entregan a un LLM, sino como **componentes de software que forman parte de una arquitectura mayor** y que deben diseñarse con los mismos criterios de calidad, mantenibilidad y flexibilidad que cualquier otro componente del sistema.

El cambio tecnológico no elimina los principios de diseño: los pone a trabajar en un contexto nuevo. Nuestro desafío es integrar la capacidad de decisión de los agentes con software diseñado de manera sólida, flexible y capaz de adaptarse a las necesidades del dominio.

Orquestación con Grafos de Estado

Si miramos en detalle el ejemplo anterior, veremos que el flujo se fue adecuando en cada momento al estado del entorno. Cada decisión tuvo una acción; cada acción, una actualización de estado, y en cada paso luego se evaluó el estado y se decidió un nuevo paso. Para conseguir actuar correctamente, el agente necesita una estructura que permita representar ese comportamiento dinámico. El problema ya no consiste solamente en definir qué herramientas están disponibles, sino en **controlar cómo se desarrolla la ejecución, qué información se conserva entre pasos y bajo qué condiciones el agente debe continuar, cambiar de camino o finalizar**.

Persistencia del diálogo vs. Amnesia del Modelo

La realidad, también, marca que obtener el resultado a través de ReAct es solamente una parte del problema, hay otras cuestiones que van más allá de la elaboración de una respuesta, pues la arquitectura debe soportar que haya usuarios conectados simultáneamente y entonces sobrellevar el diálogo con todos a la vez y sin perderse.

En un proceso de negocio real, la interacción con un usuario raramente se agota en una sola consulta aislada. Si un cliente escribe primero:

> *"Soy Textiles Norpatagonia CUIT 30-71234567-8, ¿tienen stock de tela para vaqueros en la sucursal de Córdoba?"*

… y en el turno siguiente agrega:

> *"¿Y el jersey gris también está disponible ahí?"*

… nos enfrentamos al problema de la **amnesia del modelo**. Si la arquitectura no gestiona el contexto, en cada invocación el LLM comenzará desde cero: habrá olvidado cuál era la sucursal (Córdoba), quién era el distribuidor (CUIT) y qué se había verificado previamente, e incluso saber que “tela para vaqueros” en el catálogo aparece como “denim”. Esto obligaría al sistema a solicitar los datos repetidamente o a generar respuestas descontextualizadas.

Por este motivo, independientemente de las herramientas o tecnologías que se elijan, la arquitectura de un agente inteligente requiere gestionar **dos niveles de estado y memoria**

- **Estado del bucle de ejecución (Memoria de trabajo):** Es la información que se acumula y actualiza entre los pasos internos de una misma tarea (las respuestas de las herramientas, las cotizaciones parciales, etc.).
- **Estado de la sesión (Memoria persistente):** Es la información que trasciende el turno actual y se conserva entre diferentes mensajes del usuario (la identidad del cliente, sus preferencias, el CUIT validado y el historial de decisiones).

Se necesita estructurar este comportamiento dinámico y garantizar que la información fluya sin perderse ni degradarse a través de una arquitectura capaz de controlar las decisiones del modelo mediante **ciclos explícitos y memoria compartida**.

Para materializar este patrón usaremos **LangGraph**, que nos servirá como un **cerebro organizador** de las tareas que el LLM tendrá disponibles como actuadores a fin de cumplir su labor. LangGraph permite modelar el comportamiento del agente como un **grafo de estado**, en el que cada componente representa una parte concreta de la ejecución y el estado compartido mantiene la información necesaria para tomar las decisiones siguientes.

Para que este flujo no sea rígido pero tampoco un caos impredecible, pasamos de scripts simples a **Grafos de Estado (StateGraphs)**. En esta arquitectura intervienen tres elementos fundamentales :

1. **Nodos:** representan unidades de procesamiento, como funciones de Python, llamadas al LLM o ejecución de herramientas.
2. **Aristas Condicionales (Conditional Edges):** determinan cómo se conecta un nodo con otro. Pueden abrir y cerrar “compuertas” para definir recorridos directos o decisiones condicionales según el estado actual.
3. **Estado (State):** un objeto compartido que viaja entre los nodos y contiene la información que se mantiene durante la ejecución y que puede ser utilizada por los distintos nodos para tomar decisiones.

De esta manera, el grafo proporciona la estructura que permite **materializar el ciclo ReAct** que acabamos de analizar: el modelo evalúa el estado, decide una acción, la herramienta se ejecuta, el resultado actualiza el estado y el flujo vuelve a evaluarse.

Continuidad y Delegación en Sistemas Agénticos

Cuando el ciclo de ejecución de un agente termina y el usuario continúa la conversación (el paso del Turno 1 al Turno 2), nos enfrentamos al problema de la amnesia del modelo. Para un agente sin memoria, cada interacción es un borrón y cuenta nueva, lo que lo obliga a repreguntar datos de forma redundante (como el CUIT o la sucursal de preferencia).

Para resolverlo, la arquitectura organiza la información en dos niveles lógicos fundamentales. Volvemos a verlo desde otra perspectiva:

- **Memoria de Trabajo**: La información volátil que el agente necesita retener únicamente *mientras está resolviendo una tarea específica* (los pasos inmediatos del ciclo ReAct).
- **Memoria de la Sesión (Conversacional)**: El contexto estable que debe conservarse *entre diferentes mensajes del usuario* para dar continuidad y coherencia histórica a la interacción (como la identidad del distribuidor y sus preferencias).

Para que este estado no se limite a una lista de mensajes, diseñamos un **Estado Enriquecido**. Este funciona como un contexto operativo que transporta variables de negocio clave (CUIT validado, sucursal, categorías de cliente). En la puerta de entrada, un **Nodo de Extracción de Contexto** (utilizando la extracción estructurada que aprendimos en **Integración de Modelos y Consumo de APIs**) se encarga de transformar el lenguaje natural del cliente en estas variables operativas estructuradas.

Dado que múltiples nodos y herramientas escribirán en este estado compartido a lo largo del flujo, la arquitectura debe prever mecanismos de acumulación de información (en lugar de sobrescritura automática). Al indicarle al sistema que cada nuevo paso o mensaje debe agregarse de manera incremental (lo que en el código de LangGraph implementaremos mediante **funciones de reducción** o ***reducers***), preservamos intacta la memoria y la trazabilidad de la ejecución del agente.

Control de sesión

Hay que recordar que un entorno como este no está creado para un único usuario. Para que el sistema opere de forma segura en producción, este nivel de memoria debe estar unívocamente vinculado a un **identificador único de sesión** (que en el código se implementará como thread_id) y que agrupará y dará contexto a todos los datos de la sesión de un único usuario y desde su conexión a su desconexión. **Esto garantiza un aislamiento estricto y evita el riesgo crítico de que un cliente acceda accidentalmente al contexto de otro**.

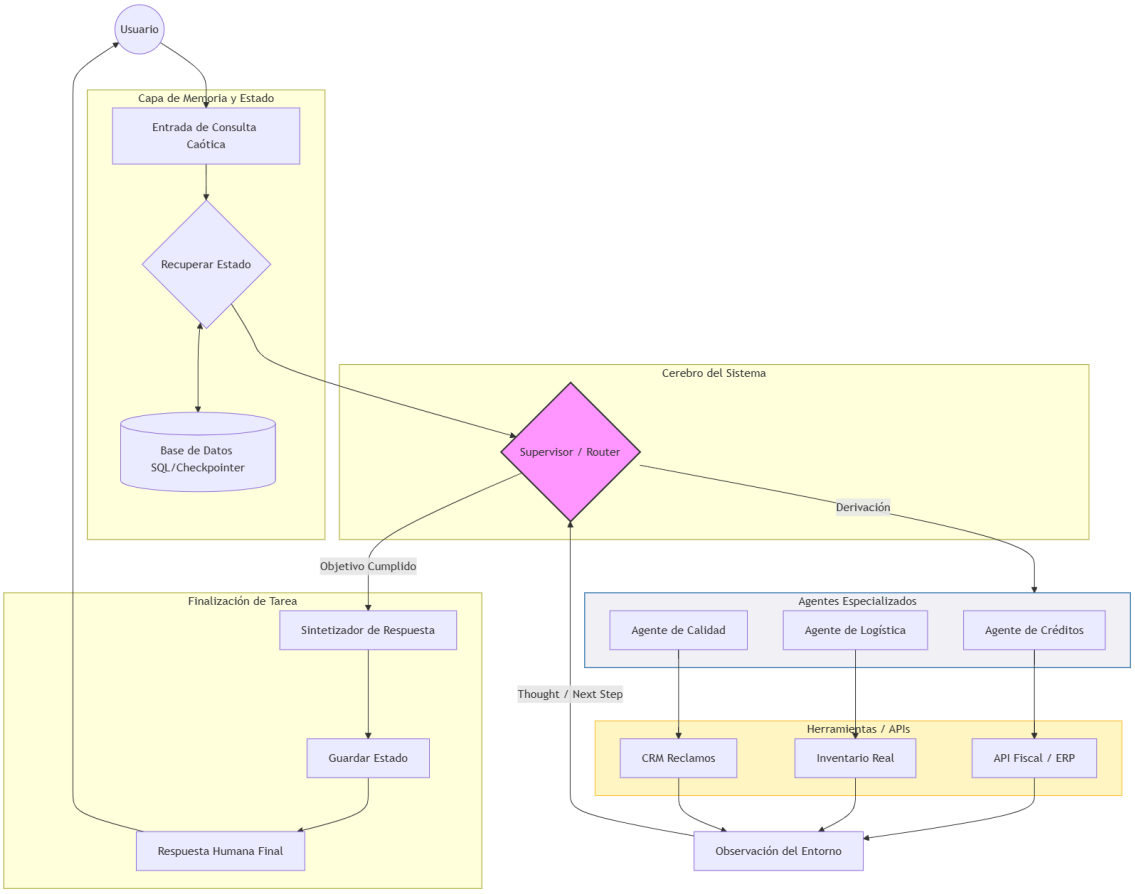
Control de la Complejidad (Multiagente y Delegación)

A medida que sumamos capacidades operativas a nuestro agente de Ortelana Textil, surge el problema de la sobrecarga de herramientas (*Tool Overload*). Concentrar

decenas de herramientas en un solo agente satura su ventana de atención y degrada drásticamente su precisión, provocando alucinaciones en la invocación de funciones.

La solución arquitectónica es la especialización y la delegación:

- Dividimos el sistema en una red de subagentes expertos (Crédito, Logística, Reclamos), logrando un aislamiento de contexto donde cada uno opera solo en su subdominio con un prompt limpio y acotado.
- Implementamos un Supervisor (orquestador) como único punto de contacto, el cual analiza el mensaje del cliente y deriva el control al experto correspondiente.
- Bajo este enfoque, el Supervisor ve a los subagentes como capacidades abstractas de alto nivel. Esto permite abstraer la complejidad, ocultando las reglas internas y herramientas de cada subdominio detrás de una delegación limpia (por ejemplo, delegar la "situación crediticia" y esperar simplemente un resultado aprobado/rechazado).



En las primeras etapas del desarrollo resolvimos la orientación del usuario mediante **clasificadores de intenciones** y **búsquedas semánticas pre-filtradas**. Aquel selector inicial operaba como un "recepcionista": leía el mensaje, asignaba una categoría rígida (*Stock*, *Crédito*, *Reclamo*) y derivaba la consulta a un pipeline informativo estático.

Al dotar al sistema de capacidad operativa, el selector tradicional resulta insuficiente. Si un distribuidor de Ortelana Textil escribe: *"Tengo un rollo fallado de Denim 12 oz, quiero devolverlo y ajustar mi límite de crédito para reservar stock de otra tela"*, un enrutador clásico se ve obligado a elegir una sola etiqueta.

El **Supervisor Multiagente** reemplaza esta lógica puntual por una **orquestación con estado**. El sistema no encasilla la frase en una categoría, sino que coordina una red de expertos: activa primero al *Agente de Reclamos* para validar la falla, transfiere el resultado al *Agente de Crédito* para procesar el ajuste y finaliza en el *Agente de Logística* para efectuar la reserva. El enrutamiento deja de ser un clasificador de texto para convertirse en un director de orquesta.

Evolución del Control de Flujo en la Arquitectura

Dimensión	Selector Primitivo en Asistentes	Supervisor Multiagente
Mecánica de Selección	Estática (Clasificación Pydantic o similitud semántica por *threshold*).	Dinámica (Grafo dirigido con aristas condicionales y evaluación continua de estado).
Manejo de Ambigüedad	Colapsa o fuerza una opción única ante consultas complejas o compuestas.	Descompone la consulta en fases y deriva secuencial o paralelamente a múltiples subagentes.
Memoria y Contexto	Evaluación puntual *single-turn* sobre el mensaje entrante.	Evaluación de estado enriquecido con persistencia a corto y largo plazo (LangGraph).
Resultado de la Ruta	Devuelve una plantilla de prompt o ejecuta una cadena RAG informativa.	Invoca subagentes que ejecutan herramientas (*Tools*) y modifican el entorno del negocio.

Con esto, ya estamos preparados para encarar soluciones más complejas que una simple consulta, permitiendo una integración y orquestación antes impensada.

Pero nos falta un poco más:

Para llevar una solución como la de Ortelana Textil a un estándar profesional y competitivo en el mercado, debemos ir más allá del diseño lógico del grafo. El paso final del diseño sistémico nos obliga a responder tres interrogantes críticas que definirán el éxito o el fracaso de nuestra arquitectura en producción:

- ¿Cómo transformamos este grafo interactivo en un producto accesible para el negocio? En el laboratorio trabajamos dentro de un entorno controlado como Google Colab. Sin embargo, un distribuidor real no consume código Python. ¿Cómo exponemos nuestro flujo agéntico como un microservicio seguro,

stateless y de alta disponibilidad utilizando una API HTTP (FastAPI) y contenedores (Docker), garantizando al mismo tiempo que la memoria persistente en SQLite se aísle correctamente por cliente?

- ¿Cómo controlamos, auditamos y medimos lo que pasa "dentro" de la mente de nuestro agente? Cuando delegamos el flujo a decisiones dinámicas del Supervisor y sus aristas condicionales, el sistema se vuelve una caja negra. Si el bot responde incorrectamente a un cliente, ¿cómo detectamos de inmediato si el fallo ocurrió en el ruteo del Supervisor, en una alucinación del LLM o en la recuperación del sensor RAG? ¿Cómo implementamos los tres pilares de la observabilidad (logs JSON, telemetría de trazas con LangSmith y métricas operativas de latencia y costo financiero por consulta)?
- ¿Cómo blindamos el sistema ante vulnerabilidades y cómo aseguramos su evolución en el tiempo? Los prompts de control y seguridad coexisten horizontalmente con las entradas de texto del usuario. ¿Cómo evitamos de forma infalible ataques maliciosos de inyección de prompts (*Prompt Injection*) o fugas de datos confidenciales de otros distribuidores (*PII*) si el LLM no puede defenderse solo? Y fundamentalmente, dado que los catálogos cambian, el lenguaje de los clientes muta (*Data Drift*) y los modelos comerciales se actualizan sin previo aviso (*Model Drift*), ¿cómo estructuramos un ciclo de vida (AIOps) que permita versionar prompts, testear variantes de flujo (Pruebas A/B) y actualizar el conocimiento semántico continuamente sin degradar el servicio?

No se trata únicamente de lograr que la IA "piense" o "actúe"; se trata de construir la infraestructura de ingeniería necesaria para que opere de manera predecible, segura, ética y rentable en el mundo real.